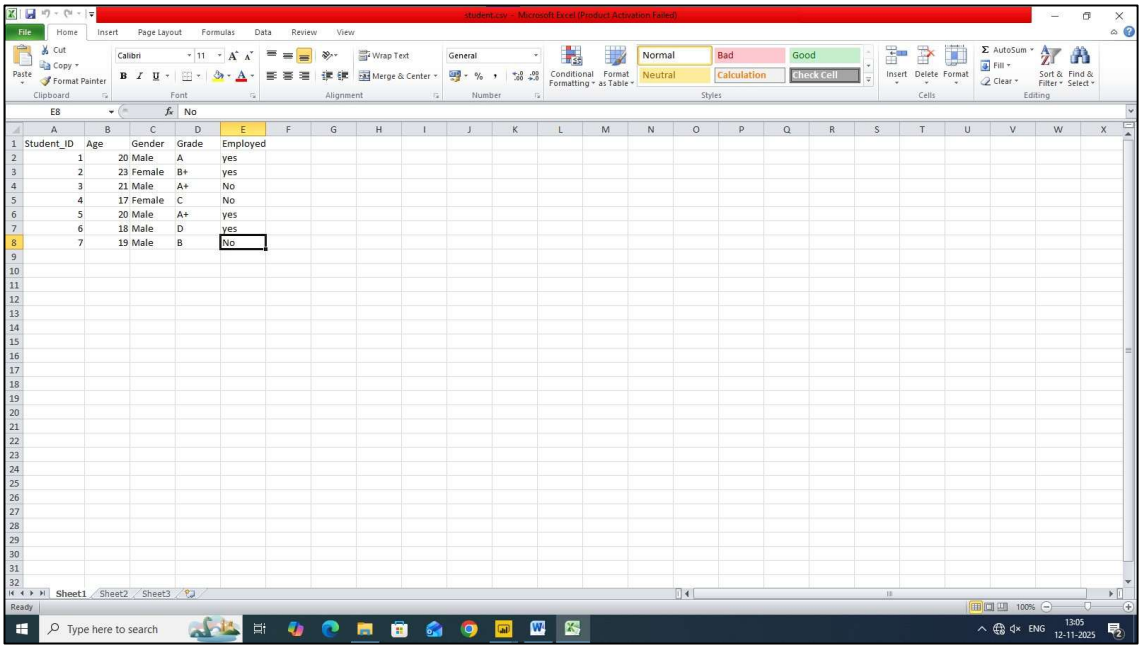
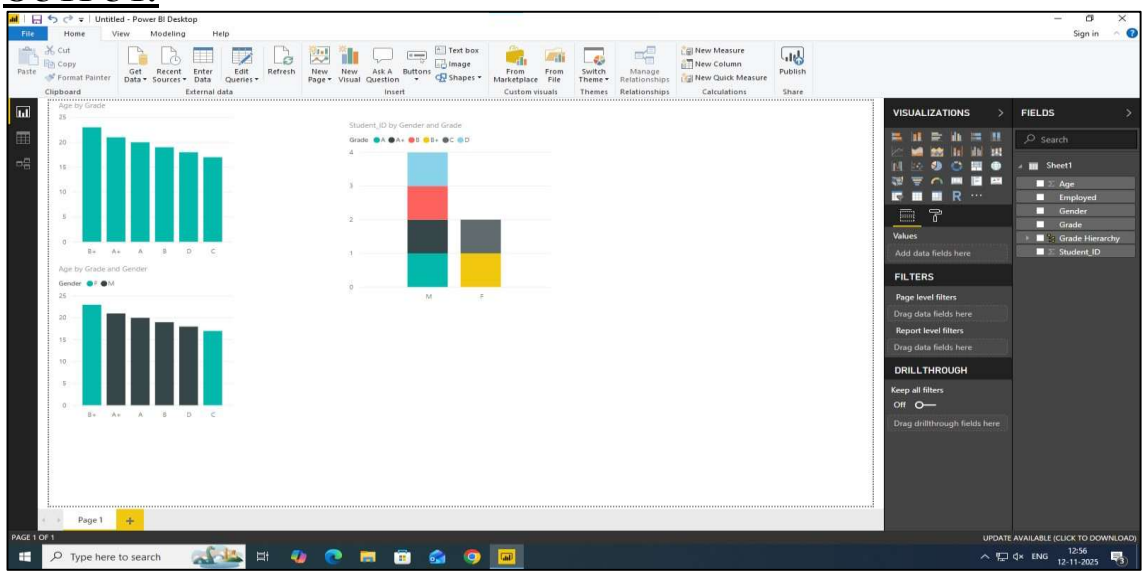
Practical No.: 1

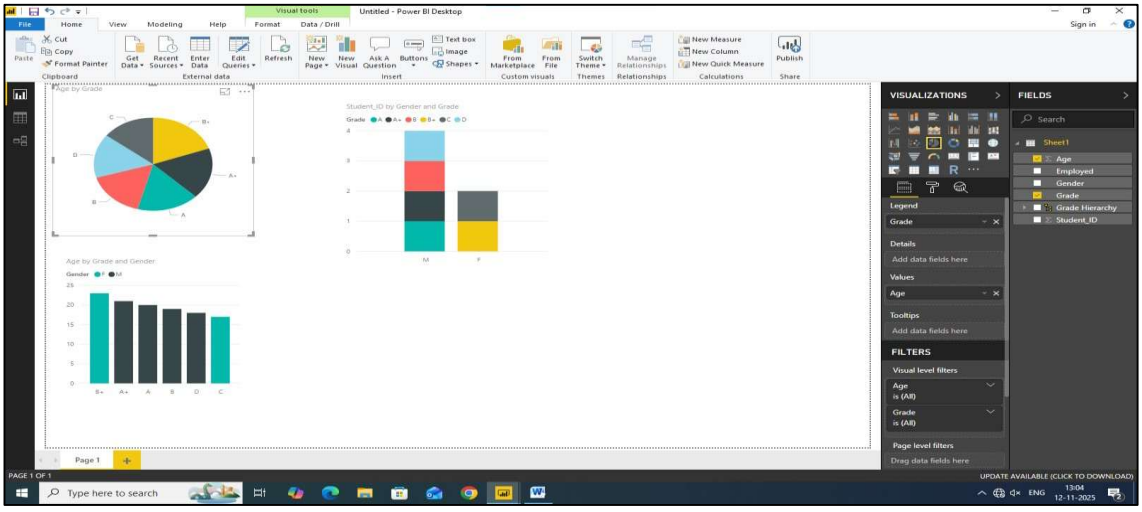
AIM: Perform different operations of extraction, transformation, and loading (ETL) processes on simple dataset using Power BI.

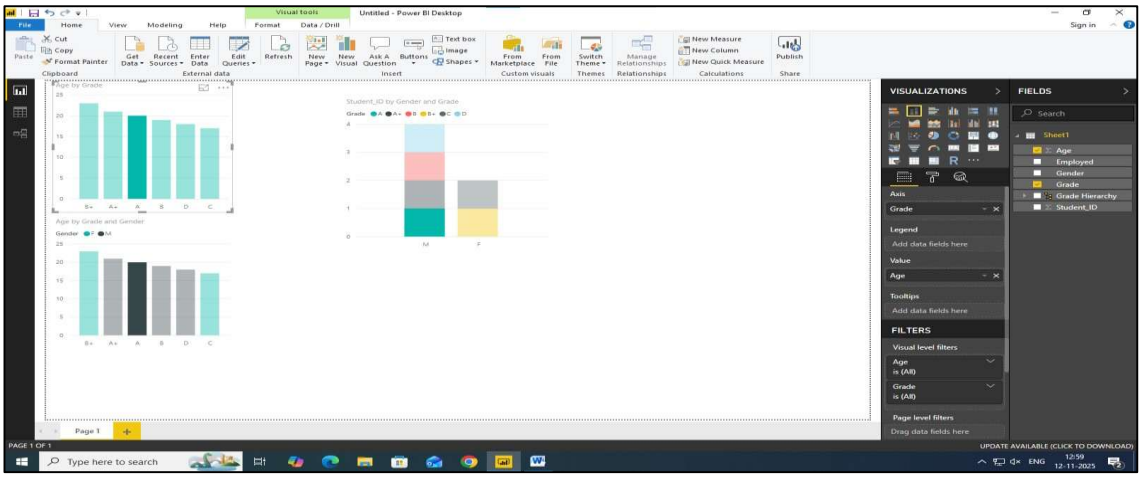


Student_ID	Age	Gender	Grade	Employed
1	20	Male	A	yes
2	23	Female	B+	yes
3	21	Male	A+	No
4	17	Female	C	No
5	20	Male	A+	yes
6	18	Male	D	yes
7	19	Male	B	No

OUTPUT:


Untitled - Power BI Desktop
Age by Grade
Student_ID by Gender and Grade
Age by Grade and Gender
VISUALIZATIONS
FIELDS
FILTERS
DRILLTHROUGH


Untitled - Power BI Desktop
Age by Grade
Student_ID by Gender and Grade
Age by Grade and Gender
VISUALIZATIONS
FIELDS
FILTERS


Untitled - Power BI Desktop
Age by Grade
Student_ID by Gender and Grade
Age by Grade and Gender
VISUALIZATIONS
FIELDS
FILTERS

Practical No.: 2

AIM: Integrate data from merging and transforming datasets using python pandas library and data manipulation techniques.

```
import pandas as pd

df1=pd.read_csv("student.csv")

df2=pd.read_csv("marks.csv")

df1.head(5) df2.head(5)
```

[5]:

	Student_ID	Mark	City
0	1	1	Mumbai
1	2	2	Virar
2	3	3	Vasai
3	4	4	Pune
4	5	5	Thane

```
df=pd.merge(df1,df2,on='Student_ID')
df.head(5)
```

	Student_ID	Age	Gender	Grade	Employed	Mark	City
0	1	20	Male	A	yes	1	Mumbai
1	2	23	Female	B+	yes	2	Virar
2	3	21	Male	A+	No	3	Vasai
3	4	17	Female	C	No	4	Pune
4	5	20	Male	A+	yes	5	Thane

```
sorted=df1.sort_values(by=['Age'])

display(sorted)
```

	Student_ID	Age	Gender	Grade	Employed
3	4	17	Female	C	No
5	6	18	Male	D	yes
6	7	19	Male	B	No
0	1	20	Male	A	yes
4	5	20	Male	A+	yes
2	3	21	Male	A+	No
1	2	23	Female	B+	yes

```
sorted=df1.filter(['Age', 'Gender', 'Grade']) display(show)
```

	Age	Gender	Grade
0	20	Male	A
1	23	Female	B+
2	21	Male	A+
3	17	Female	C
4	20	Male	A+
5	18	Male	D
6	19	Male	B

display(df1.duplicated())

dups_removed=df1.drop_duplicates() display(dups_removed)

```
0    False
1    False
2    False
3    False
4    False
5    False
6    False
dtype: bool
```

	Student_ID	Age	Gender	Grade	Employed
0	1	20	Male	A	yes
1	2	23	Female	B+	yes
2	3	21	Male	A+	No
3	4	17	Female	C	No
4	5	20	Male	A+	yes
5	6	18	Male	D	yes
6	7	19	Male	B	No

renamed=df1.rename(columns={'Employed':'Employed_status'})

display(renamed)

OUTPUT:

	Student_ID	Age	Gender	Grade	Employed_status
0	1	20	Male	A	yes
1	2	23	Female	B+	yes
2	3	21	Male	A+	No
3	4	17	Female	C	No
4	5	20	Male	A+	yes
5	6	18	Male	D	yes
6	7	19	Male	B	No

Practical No.: 3

AIM: Apply feature selection technique like variance thresholding and correlation analysis using python's scikit-learn library to reduce dimensionality in a dataset.

```
import numpy as np import
pandas as pd

from sklearn.linear_model import LassoCV from
sklearn.feature_selection import SelectFromModel from
sklearn.model_selection import train_test_split from
sklearn.metrics import classification_report from
sklearn.datasets import load_breast_cancer from
sklearn.ensemble import RandomForestClassifier

import matplotlib.pyplot as plt import
seaborn as sns

# Load the Breast Cancer databaset cancer=load_breast_cancer()
X,y=cancer.data, cancer.target

#split the data
X_train, X_test, y_train, y_test=train_test_split(X,y,test_size=0.2, random_state=42)

#fit LassoCV Model lasso_cv=LassoCV(cv=5)
lasso_cv.fit(X_train, y_train)
```

```
LassoCV
LassoCV(cv=5)
```

```
# feature selection sfm=SelectFromModel(lasso_cv,
prefit=True)
X_train_selected=sfm.transform(X_train)
X_test_selected=sfm.transform(X_test)
```

```
#train a random forest classifier using the slected feature
model=RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train_selected, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

```
#evalute the model y_pred=model.predict=(X_test_selected)
print(classification_report,(y_test, y_pred))
```

```
<function classification_report at 0x0000020D2A3BFE20> (array([1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1,
       0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
       0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1,
       0, 1, 1, 0]), array([[ 481.9 ,   24.64,   96.05,  677.9 ],
       [1130.  ,   26.58,  165.9 , 1866.  ],
       [ 748.9 ,   26.  ,  124.9 , 1156.  ],
       [ 467.8 ,   22.91,   89.61,  515.8 ],
       [ 402.9 ,   19.68,   78.78,  457.8 ],
       [1265.  ,   39.42,  184.6 , 1821.  ],
       [1482.  ,   25.8 ,  195.  , 2227.  ],
       [ 955.1 ,   19.52,  134.9 , 1227.  ],
       [ 520.  ,   23.19,   96.66,  614.9 ],
       [ 599.4 ,   21.8 ,  101.2 ,  718.9 ],
       [ 537.9 ,   34.23,   91.29,  632.9 ],
       [ 886.3 ,   26.3 ,  130.7 , 1260.  ],
       [ 690.2 ,   25.82,  102.3 ,  782.1 ],
```

```
#Analyze selected feature and their importance
```

```
selected_feature_indices=np.where(sfm.get_support())[0]
selected_features=cancer.feature_names[selected_feature_indices]
```

coefficients=lasso_cv.coef_ print("Selected Feature:", selected_features)

print("Feature Coefficients:", coefficients)

OUTPUT:

```
Selected Feature: ['mean area' 'worst texture' 'worst perimeter' 'worst area']
Feature Coefficients: [-0.         -0.         -0.          0.00025492 -0.         -0.
 -0.         -0.         -0.         -0.         -0.         -0.
 -0.         -0.         -0.         -0.         -0.         -0.
 -0.         -0.         -0.         -0.00999371 -0.01746145  0.00026856
 -0.         -0.         -0.         -0.         -0.         -0.        ]
```

Practical No.: 4

AIM: Build a decision tree classifier using python's scikit learn library to predict customer churn based on historical data.

```
import pandas as pd import
numpy as np import seaborn
as sns import
matplotlib.pyplot as plt
%matplotlib inline
```

```
df=pd.read_csv("Bank_Churn.csv")
df.head()
```

	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
0	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	1	101348.88	1
1	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	1	112542.58	0
2	15619304	Onio	502	France	Female	42	8	159660.80	3	1	0	113931.57	1
3	15701354	Boni	699	France	Female	39	1	0.00	2	0	0	93826.63	0
4	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	1	79084.10	0

```
df=df.drop(['CustomerId','HasCrCard','Age'], axis=1) df.head()
```

	Surname	CreditScore	Geography	Gender	Tenure	Balance	NumOfProducts	IsActiveMember	EstimatedSalary	Exited
0	Hargrave	619	France	Female	2	0.00	1	1	101348.88	1
1	Hill	608	Spain	Female	1	83807.86	1	1	112542.58	0
2	Onio	502	France	Female	8	159660.80	3	0	113931.57	1
3	Boni	699	France	Female	1	0.00	2	0	93826.63	0
4	Mitchell	850	Spain	Female	2	125510.82	1	1	79084.10	0

```
df.info() df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Surname          10000 non-null  object
 1   CreditScore      10000 non-null  int64
 2   Geography        10000 non-null  object
 3   Gender           10000 non-null  object
 4   Tenure           10000 non-null  int64
 5   Balance          10000 non-null  float64
 6   NumOfProducts    10000 non-null  int64
 7   IsActiveMember   10000 non-null  int64
 8   EstimatedSalary  10000 non-null  float64
 9   Exited           10000 non-null  int64
dtypes: float64(2), int64(5), object(3)
memory usage: 781.4+ KB
```

	CreditScore	Tenure	Balance	NumOfProducts	IsActiveMember	EstimatedSalary	Exited
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	650.528800	5.012800	76485.889288	1.530200	0.515100	100090.239881	0.203700
std	96.653299	2.892174	62397.405202	0.581654	0.499797	57510.492818	0.402769
min	350.000000	0.000000	0.000000	1.000000	0.000000	11.580000	0.000000
25%	584.000000	3.000000	0.000000	1.000000	0.000000	51002.110000	0.000000
50%	652.000000	5.000000	97198.540000	1.000000	1.000000	100193.915000	0.000000
75%	718.000000	7.000000	127644.240000	2.000000	1.000000	149388.247500	0.000000
max	850.000000	10.000000	250898.090000	4.000000	1.000000	199992.480000	1.000000

df.isna().sum()

df.columns

```
Index(['Surname', 'CreditScore', 'Geography', 'Gender', 'Tenure', 'Balance',
       'NumOfProducts', 'IsActiveMember', 'EstimatedSalary', 'Exited'],
      dtype='object')
```

numcols=['CreditScore','Tenure','Balance'] fig,

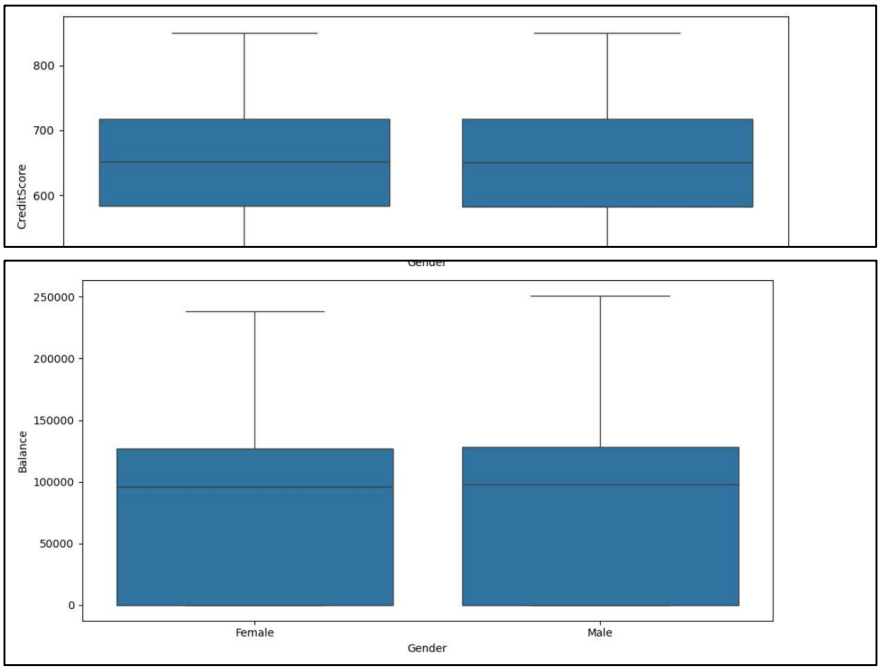
ax=plt.subplots(3,1, figsize=(10,15))

for i, subplot in zip(numcols,ax.flatten()):

    sns.boxplot(x='Gender', y=i, data=df, ax=subplot)

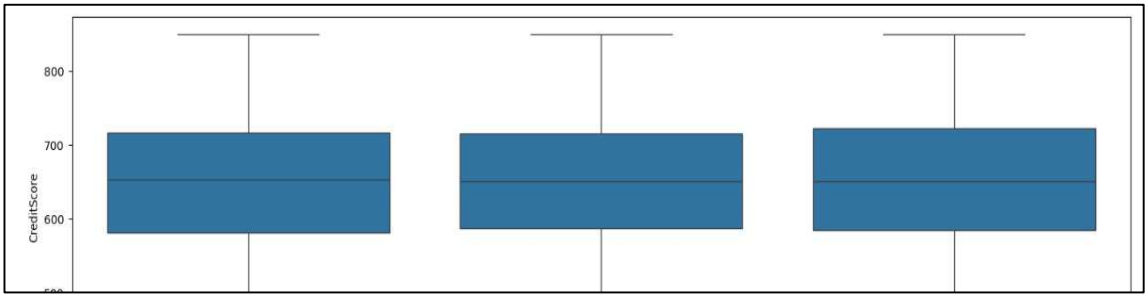
plt.tight_layout()

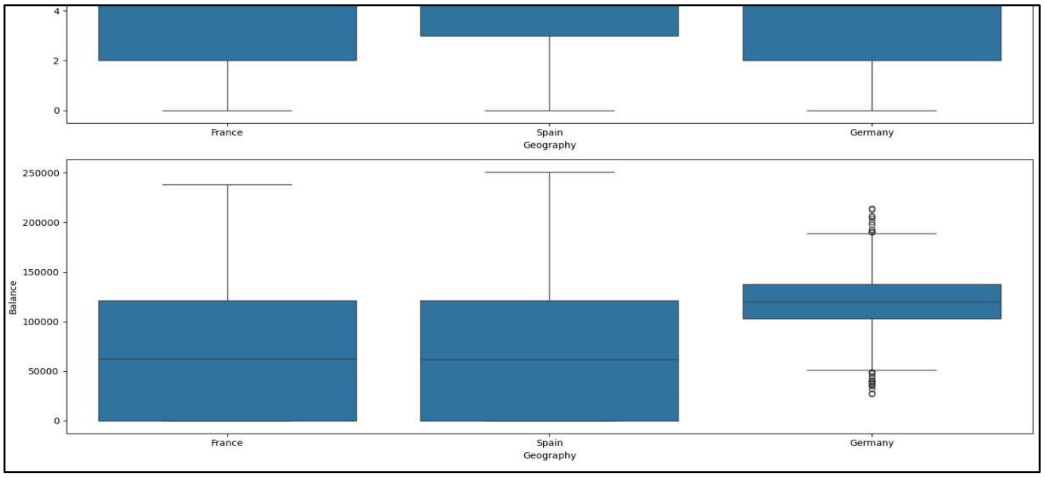
plt.show()




```
for col in numcols:     q75,
q25=np.percentile(df[col], [75,25])     iqr=q75-q25
min_val=q25-(iqr*1.5)     max_val=q75+(iqr*1.5)
df=df[(df[col]>=min_val)& (df[col]<=max_val)]

fig, ax =plt.subplots(3,1, figsize=(15,15))

for i, subplot in zip(numcols, ax.flatten()):
    sns.boxplot(x='Geography', y=i, data=df, ax=subplot)

plt.tight_layout()
```

plt.show()





from sklearn.preprocessing import LabelEncoder

le=LabelEncoder()

df['Geography']=le.fit_transform(df['Geography'])

df['Gender']=le.fit_transform(df['Gender']) le.classes_

df.head()

	Surname	CreditScore	Geography	Gender	Tenure	Balance	NumOfProducts	IsActiveMember	EstimatedSalary	Exited
0	Hargrave	619	0	0	2	0.00	1	1	101348.88	1
1	Hill	608	2	0	1	83807.86	1	1	112542.58	0
2	Onio	502	0	0	8	159660.80	3	0	113931.57	1
3	Boni	699	0	0	1	0.00	2	0	93826.63	0
4	Mitchell	850	2	0	2	125510.82	1	1	79084.10	0

from sklearn.tree import DecisionTreeClassifier from

sklearn.model_selection import train_test_split from

sklearn.preprocessing import LabelEncoder from

sklearn.metrics import mean_squared_error, r2_score

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, f1_score dtree=DecisionTreeClassifier()

df.head()

	Surname	CreditScore	Geography	Gender	Tenure	Balance	NumOfProducts	IsActiveMember	EstimatedSalary	Exited
0	Hargrave	619	0	0	2	0.00	1	1	101348.88	1
1	Hill	608	2	0	1	83807.86	1	1	112542.58	0
2	Onio	502	0	0	8	159660.80	3	0	113931.57	1
3	Boni	699	0	0	1	0.00	2	0	93826.63	0
4	Mitchell	850	2	0	2	125510.82	1	1	79084.10	0

x=df.drop('CreditScore', axis=1) y=df['CreditScore']

X_train, X_test, y_train, y_test=train_test_split(X,y, test_size=0.3, random_state=101)

dtree.fit(X_train, y_train)

dtree_predict=dtree.predict(X_test) print(f"Classification Report:/n, {classification_report(y_test, dtree_predict)}") print("Accuracy (Test set):%2f" % accuracy_score(y_test,dtree_predict)) f1=f1_score(y_test, dtree_predict, average='weighted') print(f"F1-score (Test set)

Practical No.: 5

AIM: Implement Naive Bayes Classifier in python using Scikit learn to classify emails as spam or no spam based on their content.

```
import numpy as np import pandas as pd from sklearn.model_selection import
train_test_split from sklearn.naive_bayes import MultinomialNB from
sklearn.pipeline import Pipeline from sklearn.metrics import accuracy_score,
confusion_matrix, classification_report data =
pd.read_csv('Downloads/sample_spam-ham.csv') data.head()
```

	v1	v2
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar\u\u\u then Joking wif u oni\u\u\u
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

```
s= data["v1"].value_counts()
s
```

```
ham     83
spam    17
Name: v1, dtype: int64
```

```
text=[] for i in
range(len(data)):
   ln=data["v2"][i]
   line=""     for i in ln:
if(i=='&lt;#&gt;'):
```

```
        break        line=line+i
line = line.replace("fun","")
text.append(line)
data['v2']=text
data.columns=["v1","v2"]
data.head(10)
```

	v1	v2
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar\u\u\u then Joking wif u oni\u\u\u
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...
5	spam	FreeMsg Hey there darling it's been 3 week's n...
6	ham	Even my brother is not like to speak with me. ...
7	ham	As per your request 'Melle Melle (Oru Minnamin...
8	spam	WINNER!! As a valued network customer you have...
9	spam	Had your mobile 11 months or more? U R entitle...

```
x_train, x_test, y_train, y_test=train_test_split(data['v2'],data['v1'],test_size=0.3)
from sklearn.feature_extraction.text import CountVectorizer Vectorizer =
CountVectorizer() count=Vectorizer.fit_transform(x_train.values)
MultinomialNB()
```

```
MultinomialNB()
```

```
spam_detection =MultinomialNB() targets
= y_train.values
spam_detection.fit(count,targets)
```

```
MultinomialNB()
```

```
y_predict = spam_detection.predict(Vectorizer.transform(x_test))
accuracy_score(y_test, y_predict)
```

```
0.9666666666666667
```

```
print(classification_report(y_test,y_predict))
```

OUTPUT:

```
              precision    recall  f1-score   support

         ham       0.96      1.00      0.98        25
        spam       1.00      0.80      0.89         5

    accuracy                           0.97        30
   macro avg       0.98      0.90      0.93        30
weighted avg       0.97      0.97      0.97        30
```

Practical No.: 6

AIM: Implement a linear regression method to make prediction based on sample datasets using python.

```
import numpy as np import pandas as pd
from scipy import stats
data=pd.read_csv("Downloads/data.csv")
data
```

	Unnamed	data	tcs	wipro
0	0	01-01-2026	100	88
1	1	02-01-2026	97	89
2	2	03-01-2026	88	99
3	3	04-01-2026	83	103

```
x=data['tcs']
y=data['wipro']
n=stats.linregress(x,y) n
```

```
LinregressResult(slope=-0.9354838709677419, intercept=180.81451612903226, rvalue=-0.9939855354094747, pvalue=0.0060144645905253
49, stderr=0.07287871888024747, intercept_stderr=6.723234629872086)
```

```
s=n.slope print("Slope",s)
i=n.intercept
print("Intercept",i)
```

```
Slope -0.9354838709677419
Intercept 180.81451612903226
```

```
pred_y=s*x+i print(pred_y)
```

```
0     87.266129
1     90.072581
2     98.491935
3    103.169355
Name: tcs, dtype: float64
```

r_squared=n.rvalue

print(r_squared)

OUTPUT:

```
-0.9939855354094747
```

Practical No.: 7

AIM: Implement logistic regression method to make prediction based on the sample data set using python.

```
import pandas as pd from sklearn.model_selection
import train_test_split from sklearn.linear_model
import LogisticRegression from sklearn.metrics
import accuracy_score data=pd.read_csv("iris
(1).csv")
data
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Virginica
146	6.3	2.5	5.0	1.9	Virginica
147	6.5	3.0	5.2	2.0	Virginica
148	6.2	3.4	5.4	2.3	Virginica
149	5.9	3.0	5.1	1.8	Virginica

150 rows × 5 columns

```
data.head(5)
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

data.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

data.describe()

	sepal.length	sepal.width	petal.length	petal.width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

data.columns

```
Index(['sepal.length', 'sepal.width', 'petal.length', 'petal.width',
       'variety'],
      dtype='object')
```

X= data[['sepal.length','sepal.width','petal.length','petal.width']]

Y=data['variety']

X_train, X_test, Y_train, Y_test= train_test_split(X,Y,test_size=0.3,random_state=0)

model=LogisticRegression() model.fit(X_train, Y_train)

```
LogisticRegression()
```

prediction = model.predict(X_test)
prediction

```
array(['Virginica', 'Versicolor', 'Setosa', 'Virginica', 'Setosa',
       'Virginica', 'Setosa', 'Versicolor', 'Versicolor', 'Versicolor',
       'Virginica', 'Versicolor', 'Versicolor', 'Versicolor',
       'Versicolor', 'Setosa', 'Versicolor', 'Versicolor', 'Setosa',
       'Setosa', 'Virginica', 'Versicolor', 'Setosa', 'Setosa',
       'Virginica', 'Setosa', 'Setosa', 'Versicolor', 'Versicolor',
       'Setosa', 'Virginica', 'Versicolor', 'Setosa', 'Virginica',
       'Virginica', 'Versicolor', 'Setosa', 'Virginica', 'Versicolor',
       'Versicolor', 'Virginica', 'Setosa', 'Virginica', 'Setosa',
       'Setosa'], dtype=object)
```

accuracy=accuracy_score(Y_test,prediction)

accuracy

OUTPUT:

```
0.9777777777777777
```

Practical No.: 8

AIM: Implement K-means clustering algorithm in python using scikit learn to group customers based on their purchasing behavior.

import pandas as pd from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler from

matplotlib import pyplot as plt import seaborn as sns

from sklearn.decomposition import PCA from

mpl_toolkits.mplot3d import Axes3D

data=pd.read_csv("Downloads/online-retail-dataset.csv")

data

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/2010 8:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
...	...	...	...	...	...	...	...	...
541904	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	12/9/2011 12:50	0.85	12680.0	France
541905	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	12/9/2011 12:50	2.10	12680.0	France
541906	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4	12/9/2011 12:50	4.15	12680.0	France
541907	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4	12/9/2011 12:50	4.15	12680.0	France
541908	581587	22138	BAKING SET 9 PIECE RETROSPOT	3	12/9/2011 12:50	4.95	12680.0	France

541909 rows × 8 columns

data.describe()

	Quantity	UnitPrice	CustomerID
count	541909.000000	541909.000000	406829.000000
mean	9.552250	4.611114	15287.690570
std	218.081158	96.759853	1713.600303
min	-80995.000000	-11062.060000	12346.000000
25%	1.000000	1.250000	13953.000000
50%	3.000000	2.080000	15152.000000
75%	10.000000	4.130000	16791.000000
max	80995.000000	38970.000000	18287.000000

data.dropna(inplace=True) data.isna().sum()

data.info()

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 406829 entries, 0 to 541908
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   InvoiceNo    406829 non-null  object
 1   StockCode    406829 non-null  object
 2   Description  406829 non-null  object
 3   Quantity     406829 non-null  int64
 4   InvoiceDate  406829 non-null  object
 5   UnitPrice    406829 non-null  float64
 6   CustomerID   406829 non-null  float64
 7   Country      406829 non-null  object
dtypes: float64(2), int64(1), object(5)
memory usage: 27.9+ MB
```

data.columns

```
Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity', 'InvoiceDate',
       'UnitPrice', 'CustomerID', 'Country'],
      dtype='object')
```

data['Total_Amount']=data['Quantity']*data['UnitPrice']

data['Total_Amount']

OUTPUT:

```
0         15.30
1         20.34
2         22.00
3         20.34
4         20.34
          ...
541904    10.20
541905    12.60
541906    16.60
541907    16.60
541908    14.85
Name: Total_Amount, Length: 406829, dtype: float64
```

Practical No.: 9

AIM: Implement the Apriori algorithm in python to min frequent itemset from a retail transaction dataset and extract association rules.

```
!pip install mlxtend

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: mlxtend in c:\users\student\appdata\roaming\python\python39\site-packages (0.23.4)
Requirement already satisfied: scipy>=1.2.1 in c:\programdata\anaconda3\lib\site-packages (from mlxtend) (1.9.1)
Requirement already satisfied: scikit-learn>=1.3.1 in c:\users\student\appdata\roaming\python\python39\site-packages (from mlxt
end) (1.6.1)
Requirement already satisfied: matplotlib>=3.0.0 in c:\programdata\anaconda3\lib\site-packages (from mlxtend) (3.5.2)
Requirement already satisfied: numpy>=1.16.2 in c:\programdata\anaconda3\lib\site-packages (from mlxtend) (1.21.5)
Requirement already satisfied: joblib>=0.13.2 in c:\users\student\appdata\roaming\python\python39\site-packages (from mlxtend)
(1.5.3)
Requirement already satisfied: pandas>=0.24.2 in c:\programdata\anaconda3\lib\site-packages (from mlxtend) (1.4.4)
Requirement already satisfied: cycler>=0.10 in c:\programdata\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (0.
11.0)
Requirement already satisfied: packaging>=20.0 in c:\programdata\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxtend)
(21.3)
Requirement already satisfied: python-dateutil>=2.7 in c:\programdata\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxt
end) (2.8.2)
Requirement already satisfied: fonttools>=4.22.0 in c:\programdata\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxten
d) (4.25.0)
Requirement already satisfied: pillow>=6.2.0 in c:\programdata\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxtend)
(9.2.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\programdata\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxten
d) (1.4.2)
Requirement already satisfied: pyparsing>=2.2.1 in c:\programdata\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxtend)
(3.0.9)
Requirement already satisfied: pytz>=2020.1 in c:\programdata\anaconda3\lib\site-packages (from pandas>=0.24.2->mlxtend) (2022.
1)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\student\appdata\roaming\python\python39\site-packages (from sci
kit-learn>=1.3.1->mlxtend) (3.6.0)
Requirement already satisfied: six>=1.5 in c:\programdata\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib>=
3.0.0->mlxtend) (1.16.0)
```

import numpy as np

import pandas as pd from mlxtend.frequent_patterns import apriori,

association_rules data=pd.read_csv("Downloads/online-retail-

dataset.csv",encoding='unicode_escape') data.head()

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/2010 8:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	3.39	17850.0	United Kingdom

data.Country.unique()

```
array(['United Kingdom', 'France', 'Australia', 'Netherlands', 'Germany',
       'Norway', 'EIRE', 'Switzerland', 'Spain', 'Poland', 'Portugal',
       'Italy', 'Belgium', 'Lithuania', 'Japan', 'Iceland',
       'Channel Islands', 'Denmark', 'Cyprus', 'Sweden', 'Austria',
       'Israel', 'Finland', 'Bahrain', 'Greece', 'Hong Kong', 'Singapore',
       'Lebanon', 'United Arab Emirates', 'Saudi Arabia',
       'Czech Republic', 'Canada', 'Unspecified', 'Brazil', 'USA',
       'European Community', 'Malta', 'RSA'], dtype=object)
```

```
data['Description']=data['Description'].str.strip()
data.dropna(axis=0,subset=['InvoiceNo'],inplace=True)
data['InvoiceNo']=data['InvoiceNo'].astype('str')
data=data[~data['InvoiceNo'].str.contains('C')]
basket_France=(data[data['Country']=="France"]
          .groupby(['InvoiceNo','Description'])['Quantity']
           .sum().unstack().reset_index().fillna(0)
           .set_index('InvoiceNo'))
basket_UK=(data[data['Country']=="United Kingdom"]
          .groupby(['InvoiceNo','Description'])['Quantity']
           .sum().unstack().reset_index().fillna(0)
           .set_index('InvoiceNo'))
basket_Por=(data[data['Country']=="Portugal"]
          .groupby(['InvoiceNo','Description'])['Quantity']
           .sum().unstack().reset_index().fillna(0)
           .set_index('InvoiceNo'))
basket_Sweden=(data[data['Country']=="Sweden"]
          .groupby(['InvoiceNo','Description'])['Quantity']
           .sum().unstack().reset_index().fillna(0)
           .set_index('InvoiceNo'))
def hot_encode(X):    if X <=0:
return 0    if X >=1:        return
1


basket_encoded=basket_France.applymap(hot_encode)
basket_France=basket_encoded


basket_encoded=basket_UK.applymap(hot_encode)
basket_UK=basket_encoded
```

```
basket_encoded=basket_Por.applymap(hot_encode)
basket_Por=basket_encoded

basket_encoded=basket_Sweden.applymap(hot_encode)
basket_Sweden=basket_encoded frq_items=apriori(basket_France,
min_support=0.05, use_colnames=True)
rules=association_rules(frq_items, metric="lift", min_threshold=1)
rules=rules.sort_values(['confidence','lift'],ascending=[False,False])
print(rules.head())
```

OUTPUT:

```
                                        antecedents  \
44                        (JUMBO BAG WOODLAND ANIMALS)
258  (PLASTERS IN TIN CIRCUS PARADE, RED TOADSTOOL ...
272  (RED TOADSTOOL LED NIGHT LIGHT, PLASTERS IN TI...
301  (SET/20 RED RETROSPOT PAPER NAPKINS, SET/6 RED...
302  (SET/20 RED RETROSPOT PAPER NAPKINS, SET/6 RED...

                        consequents  antecedent support  consequent support  \
44                        (POSTAGE)            0.076531            0.765306
258                       (POSTAGE)            0.051020            0.765306
272                       (POSTAGE)            0.053571            0.765306
301  (SET/6 RED SPOTTY PAPER PLATES)            0.102041            0.127551
302    (SET/6 RED SPOTTY PAPER CUPS)            0.102041            0.137755

      support  confidence      lift  representativity  leverage  conviction  \
44   0.076531       1.000  1.306667               1.0  0.017961         inf
258  0.051020       1.000  1.306667               1.0  0.011974         inf
272  0.053571       1.000  1.306667               1.0  0.012573         inf
301  0.099490       0.975  7.644000               1.0  0.086474   34.897959
302  0.099490       0.975  7.077778               1.0  0.085433   34.489796

     zhangs_metric   jaccard  certainty  kulczynski
44        0.254144  0.100000   1.000000    0.550000
258       0.247312  0.066667   1.000000    0.533333
272       0.247978  0.070000   1.000000    0.535000
301       0.967949  0.764706   0.971345    0.877500
302       0.956294  0.709091   0.971006    0.848611
```

```
C:\Users\student\AppData\Roaming\Python\Python39\site-packages\mlxtend\frequent_patterns\fpcommon.py:161: DeprecationWarning: D
ataFrames with non-bool types result in worse computationalperformance and their support might be discontinued in the future.Pl
ease use a DataFrame with bool type
  warnings.warn(
```